arXiv:2411.14436v1 [cs.AR] 4 Nov 2024

AssertLLM: Generating Hardware Verification Assertions from Design Specifications via Multi-LLMs

Zhiyuan Yan[1*], Wenji Fang[2*], Mengming Li[2], Min Li[3], Shang Liu[2], Zhiyao Xie[2†], Hongce Zhang[1,2†]
[1]Hong Kong University of Science and Technology (Guangzhou)
[2]Hong Kong University of Science and Technology
[3]Huawei Technologies Co., Ltd.

ABSTRACT

Assertion-based verification (ABV) is a critical method to ensure logic designs comply with their architectural specifications. ABV requires assertions, which are generally converted from specifications through human interpretation by verification engineers. Existing methods for generating assertions from specification documents are limited to sentences extracted by engineers, discouraging their practical applications. In this work, we present AssertLLM, an automatic assertion generation framework that processes complete specification documents. AssertLLM can generate assertions from both natural language and waveform diagrams in specification files. It first converts unstructured specification sentences and waveforms into structured descriptions using natural language templates. Then, a customized Large Language Model (LLM) generates the final assertions based on these descriptions. Our evaluation demonstrates that AssertLLM can generate more accurate and higher-quality assertions compared to GPT-4o and GPT-3.5.

ACM Reference Format:
Zhiyuan Yan[1*], Wenji Fang[2*], Mengming Li[2], Min Li[3], Shang Liu[2], Zhiyao Xie[2†], Hongce Zhang[1,2†]. 2025. AssertLLM: Generating Hardware Verification Assertions from Design Specifications via Multi-LLMs. In *30th Asia and South Pacific Design Automation Conference (ASPDAC '25), January 20–23, 2025, Tokyo, Japan.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3658617.3697756

1 INTRODUCTION

Hardware functional verification is critical in the VLSI design flow. It addresses the following question: whether an implementation adheres to its specification. The specifications are typically drafted in natural language by architects and then translated into RTL code by designers. Verification engineers then check the functional correctness of the RTL designs according to the specifications. In the verification process, assertion-based verification (ABV) [35] is a widely adopted technique, which utilizes assertions crafted from specifications to verify the functional behavior of RTL designs.

*These authors contributed equally to this work.
†Corresponding Authors: {hongcezh, eezhiyao}@ust.hk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASPDAC '25, January 20–23, 2025, Tokyo, Japan
© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0635-6/25/01
https://doi.org/10.1145/3658617.3697756

Table 1: Existing works on generating SVAs from natural language specifications. AssertLLM is the first work that can handle full-size specification files and generate comprehensive types of SVAs for each architectural signal.

Stage	Works	Gen. Method	NL Specification			Verification Target
			Auto Proc.	Source	Full Design	
RTL	[13, 22]	LLM-based	✗	Sentences (Engineers)	✗	Security
	[18, 26, 32]					Function (few examples)
Pre-RTL	[1, 8, 11, 14–16, 27, 38]	NLP-based	✗	Sentences (SPEC)	✓	Function (property checkers/ artificial cases)
	Ours	LLM-based	✓	Entire Doc (SPEC)	✓	Function (general designs)

ABV can be performed either by simulation or formal property verification (FPV), where assertions are often written in the form of SystemVerilog Assertions (SVAs). However, a significant challenge in ABV is the generation of sufficient, high-quality assertions. Currently, designing SVAs manually is a time-consuming and error-prone task, demanding unignorable human effort.

To address this challenge, research has focused on generating SVAs automatically, which can be mainly categorized into two types: (1) dynamic mining from simulation traces and (2) static analysis of specifications. Dynamic methods [6, 10, 33] combine simulation with static design constraint analysis but risk generating incorrect SVAs due to their reliance on potentially flawed RTL designs. Static methods utilize predefined templates [7, 25] or natural language processing (NLP)-based machine learning (ML) technologies [1, 4, 8, 11, 14–16, 27, 38]. Recently, the potential of generative AI technologies like Large Language Models (LLMs) has gained significant attention in hardware design process [5, 17, 19–21, 28, 36]. Researchers are also exploring the use of LLMs to generate hardware assertions [12, 26, 29, 31, 32].

We further categorize the existing static ML-based methods based on their application in different design phases: the RTL and pre-RTL stages. Table 1 details these ML-based SVA generation methods in both the RTL stage and the pre-RTL stage. During the RTL stage, the process typically involves using LLMs to process both human-written specification sentences and the RTL design to generate SVAs describing security or functional properties [12, 18, 26, 32]. However, similar to the dynamic methods, inaccuracies in RTL implementations could result in flawed SVAs.

Regarding the pre-RTL stage, with the specification document finalized, RTL designers proceed to implement behavioral designs satisfying the specification. Numerous studies [1, 8, 11, 14–16, 27, 38] have used NLP techniques to generate SVAs from natural language sentences, focusing on sentence-level analysis from extensive specifications. However, this approach has several limitations. It is labor-intensive, requiring manual extraction of relevant sentences. It also

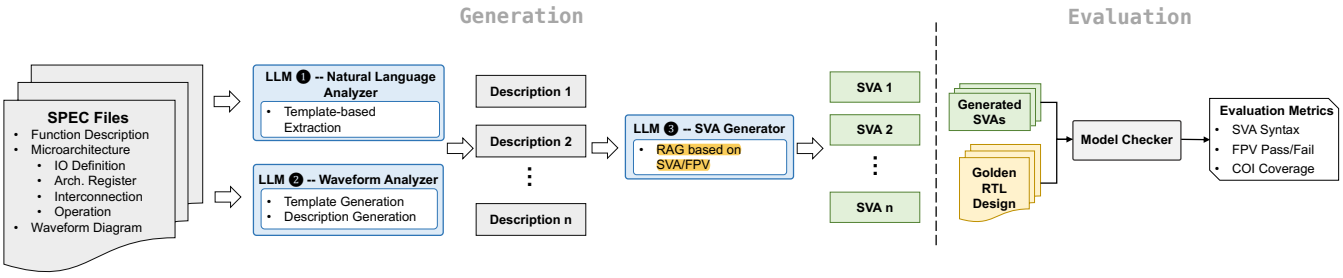


Figure 1: AssertLLM generation and evaluation workflow. AssertLLM incorporates three LLMs, each enhanced with specific techniques for the decomposed tasks: extracting structural information from natural language, extracting structural information from waveform diagrams, and translating extractions into various SVA types. The generated SVAs are further evaluated based on the golden RTL implementations using model checking tools.

struggles with diverse grammatical structures, and its evaluation often relies on design-specific checkers, limiting broader applicability. Furthermore, specifications frequently include waveform diagrams that illustrate functional behaviors across different signals. Nevertheless, there is no existing technique that can generate SVAs from these diagrams currently, representing a significant gap in the field.

Here we summarize three key challenges that currently hinder the practical application of SVA generation from specification documents:

(1) Natural language VLSI specifications are often unstructured and are hard to be directly used for assertion generation.
(2) Even with structured specifications, translating natural language into assertions remains a highly complex task, requiring both a deep understanding of the design functionality and an expertise in SVA.
(3) While waveforms commonly exist in specification documents, no current research focuses on capturing functional behaviors from waveforms and generating corresponding SVAs.

To tackle these challenges in SVA generation, in this work, we propose AssertLLM, a novel automatic assertion generation framework incorporating multiple LLMs to deal with the decomposed tasks separately. This framework is designed to process the entire specification files as they are, and automatically produce SVAs across different signals, significantly benefits both design-time bug prevention and verification-time bug detection. AssertLLM operates in three main steps. First, it processes natural language in specification documents, using an LLM to convert unstructured information into a structured format via a unified template. Second, it analyzes waveform diagrams in the specification files, employing another LLM to create natural language templates and extract waveform descriptions based on these templates. Unlike previous methods [7, 25], AssertLLM does not require an additional human input to create templates. Finally, a customized LLM translates the extracted information into SVAs. The resulting SVAs check various design aspects, including bit-width, connectivity, and functionality.

Our contributions in this work are summarized below:

- To the best of our knowledge, AssertLLM is the first automatic assertion generation method that can handle the complete specification files.
- We decompose the assertion-generation process into three key steps: extracting structural information from natural language, generating descriptions from waveform diagrams, and translating the information into SVAs. These SVAs support checks for bit-width, connectivity, and functionality.
- To demonstrate the effectiveness of AssertLLM, we conduct a comprehensive evaluation on a set of designs. The result shows that 88% of generated SVAs are evaluated to be correct both syntactically and functionally. Furthermore, these correct SVAs achieve 97% cone of influence coverage, demonstrating the high quality of the generated SVAs.

2 METHODOLOGY

2.1 Workflow Overview

Figure 1 illustrates the process of generating and evaluating SVAs for AssertLLM. Our approach integrates three LLMs to generate hardware verification assertions from comprehensive specification documents. These models perform the following tasks: 1) extract relevant information from the natural language in the specification necessary for SVA generation; 2) extract behavioral descriptions from waveform diagrams for SVA generation; 3) generate high-quality SVAs based on the previous extracted information.

In the subsequent subsections, we will detail the functionalities of each LLM of the assertion generation flow. Following this, our SVA evaluation methodology will be presented.

2.2 Natural Language Analyzer

A comprehensive natural language specification typically includes seven key sections: 1) introduction: outlines the design's concepts and features; 2) IO ports: provides detailed information for the interface; 3) registers: describes all the architecture-level registers in the design; 4) operation: explains the operational procedures for dataflow and control; 5) architecture: the high-level workflow and dataflow of the design; and 6) usage examples: offers basic usage scenarios for the design. For signals, the specification may only define critical architecture-level IO ports and registers, leaving the designers to detail internal signals for RTL implementations. 7) waveform diagram: describe behaviors for different signals.

The first step of our AssertLLM framework is to extract structured information from natural language specification documents

[Prompt for SPEC Analyzer]

- [Upload the original SPEC file] Here is the design specification file, please analyze it carefully.
- [For each signal] Please extract all the information related to the signal control register (CTR) from the spec file.

[Response from SPEC Analyzer]

The Control Register (CTR) in the I2C-Master Core specification has the following characteristics:

1. [Signal Name]: In Spec - Control Register (CTR).
2. [Description]:
   2.1. [Definition]:
      - Bit-width: 8 bits
      - Signal Type: Register (Access Type: RW - Read/Write)
   2.2. [Functionality]:
      - Bit 7: I2C core enable bit. When set to '1', the core is enabled, and when set to '0', the core is disabled.
      - Bit 6: I2C core interrupt enable bit. When set to '1', interrupts are enabled, and when set to '0', interrupts are disabled.
      - Bits 5:0: Reserved. These bits are read/write but are reserved for future use.
   2.3. [Interconnection]: This register directly controls the operation of the I2C core, enabling or disabling it and controlling interrupt behavior.
   2.4. [Additional Information]:
      - Reset Value for the Control Register is 0x00
3. [Related Signals]:
      - EN: Controls the enable state of the I2C core.
      - IEN: Controls the interrupt enable state of the I2C core.

Figure 2: Prompt and Response Example of LLM ❶ `Natural Language Analyzer`

to enable SVA generation. As we discussed in Section 1, the first key challenge of SVA generation lies in the inherent unstructured nature of the original specifications, which contain background information, functional descriptions, microarchitecture designs, and various diagrams, including dataflow and waveform, etc. Meanwhile, the existence of assertion-relevant information across different sections further complicates the direct utilization of the original specifications for SVA generation.

To address the challenge of processing original, unstructured, full-size specification documents, we utilize an LLM tailored to extract structural and relevant information for each defined signal, thereby further facilitating the SVA generation process.

Specifically, in our LLM ❶ `Natural Language Analyzer`, we first utilize system instructions to customize the LLM. The model takes the full-size specification file as the input, and the multi-modal function is employed to analyze the file containing text, tables, figures, etc. Then for each signal, the LLM is required to extract all the related information of the signal. Here, we design a unified and structured template to guide the LLM in extracting all essential signal-related information. This template contains three key components: the signal's name, its description, and the interconnection signals. We demonstrate the details of each part as follows:

- **Name**: The identifier of the signal in the specification, ensuring clear and unambiguous reference.
- **Description**: To facilitate SVA generation, we divide the descriptions into four categories, including 1) definitions such as bit-width and signal type. 2) functionality which contains all the function-related information of the target signal in the entire specification file. 3) interconnection relationship with all other signals. 4) additional information that is not included in the above three types.
- **Interconnection Signals**: A list of signals that interact or are associated with the target signal, which are essential for the subsequent assertion generation process.

Note that the extracted information is summarized across different sections of the original specification, which contains all the *textual* information needed for assertion generation. In order to incorporate specifications from the waveform, we employ another LLM to extract information from the waveform diagrams as presented in next subsection, thereby enriching the set of SVAs and potentially enhancing their overall quality.

2.3 Waveform Analyzer

In this work, we are interested in extracting design behaviors from waveforms in the specification document, where these waveforms are presented in the form of images (a two-dimensional array of pixels). Unlike those prior methods [6, 10, 33] that assume a numeric and structured waveform, such as those stored in the Value Change Dump (VCD) format, AssertLLM needs to first interpret the images of waveforms to obtain a description of behaviors. Techniques like optical character recognition (OCR) [24] may be used to convert diagrams of a fixed format into text, however, it would be hard to accommodate various waveform styles. Whereas, LLM is advantageous in its flexibility. Despite that multi-modal LLMs can take images as input and there are prior works like image captioning [2, 3] that interpret input images with human languages. However, they are not suitable for waveform interpretation.

To address this problem, we propose another LLM ❷ `Waveform Analyzer` to extract behavior descriptions from waveform diagrams. This analyzer employs natural language to describe the behavior of various signals in the waveform through a two-step process. In the first step, `Waveform Analyzer` automatically creates a set of templates suitable for describing the behaviors in the given waveform. Subsequently, `Waveform Analyzer` produces the behavioral descriptions based on these templates.

2.3.1 Template Generation. It requires significant effort for a human to write templates to describe behaviors on a single signal or among various signals on the waveform diagrams. To solve this problem, we employ prompt engineering to automate the template generation process. Fig. 3 shows the prompt and corresponding response in the step of template generation. These automatically generated templates serve as references for producing descriptions of given waveform diagrams in the subsequent step.

2.3.2 Description Generation. In this step, `Waveform Analyzer` takes our unified prompt, extracted waveform diagrams, and generated templates as inputs to produce descriptions of the behavior for the given waveform. Fig. 4 shows the corresponding prompt and response in the description generation step. By processing and analyzing the waveform diagrams and the templates, our `Waveform Analyzer` comprehensively investigates the functional information in the waveforms, thereby identifying potential relationships among different signals. For example, it can determine whether the output signal is valid when a specific signal is enabled or deduce that a signal should remain stable for a specific number of cycles when the corresponding condition is met. Furthermore, we prompt the LLM to directly use the provided templates or extend

[Prompt for Template Generation]

You are an expert in digital hardware design at the register-transfer level (RTL). Please generate a set of prompt templates. We will use these prompt templates to summarize the behaviors that can be observed in hardware waveforms, and these observations will be further utilized to generate assertions in SystemVerilog format. As a consequence, the generated templates need to describe all logic behaviors at the RTL level for a variable or across different variables. And it needs to be easily transferred to the SystemVerilog assertions by human engineers. Here are the requirements for the answer:
1. Please keep templates as brief and clear as possible.
2. Please number the templates.

[Response from Template Generation]

Here are some prompt templates for summarizing observed behaviors in hardware waveforms at the RTL level:

1.If <signal> is high, then <variable> must be low in the next cycle.
2.When <condition> occurs, <variable1> should be equal to <variable2>.
…
…
18.<variable> should remain stable for <number> cycles after <event>.
19.<variable1> must be greater than <variable2> after <event>.
20.<variable> must be equal to <expression> when <condition> is true.

Figure 3: Prompt and Response Example for template generation in LLM ❷ `Waveform Analyzer`

them if necessary. This flexibility allows the `Waveform Analyzer` to explore additional behaviors in the waveform.

2.4 Automatic Assertion Generation

For the conversion from textual descriptions to assertions, prior research has explored traditional NLP techniques for pre-RTL stages and LLM-based approaches for RTL designs, each with its limitations. NLP methods require detailed syntax and semantics analysis, limiting adaptability to sentence structure variations. LLM approaches, focusing on the RTL stage, depend on HDL code and annotations but risk generating inaccurate SVAs from unverified RTL code, potentially compromising the verification process.

To address these challenges, our work introduces the LLM ❸ `SVA Generator`, dedicated to generating assertions for each signal utilizing the previously extracted structural information.

Considering the precise syntax and writing rules inherent to SVAs and the potential for the original LLM failing to generate syntactically correct SVAs, as discussed in [26], we incorporate the Retrieval Augmented Generation (RAG) technique [37] to enhance the LLM's capability for SVA generation. This approach is enriched by a knowledge database comprising tutorials and textbooks on SVA and formal property verification [23, 30, 34], providing a robust foundation for the LLM to access and retrieve relevant SVA knowledge based on the input query, thereby enhancing the quality of the generated SVAs.

In addition to the RAG technique, we guide the `SVA Generator` with the overall architecture diagram of the design. The LLM is provided with the structured specifications extracted from the previous LLMs for each signal. Then the LLM is required to generate SVAs that adhere strictly to the specifications. We categorize SVAs into three types: 1) **width**: check if the signal bit width satisfies the specification; 2) **connectivity**: check if the signal can be correctly

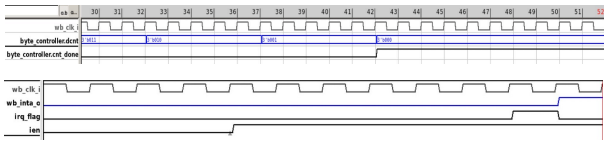
[Prompt for Description Generation]

You are an expert in digital hardware design and verification at the Register-transfer level (RTL). Your task is to analyze the behaviors from the given hardware waveform for a variable or across different variables. Please use different serial numbers to represent different behaviors from the waveform.

Furthermore, we provide the potential templates for describing behaviors. There is no need for you to use all templates. You only need to select some templates. Also, you can revise the templates if they can better describe the behavior in the waveform:
- [Upload the generated templates from Template Generator]

…
…

I will provide the waveform you need to analyze in the following blocks. Please do use the English to answer the question.
- [Upload the waveform diagram]



[Response from Description Generation]

Here are some typical behaviors you might observe in a waveform for a variable or across different variables:

1. When byte_controller.dcnt is 3'b000, byte_controller.cnt_done is set to high.
2. byte_controller.dcnt decrements its value in each clock cycle until it reaches 3'b000.
3. byte_controller.cnt_done remains low until byte_controller.dcnt reaches 3'b000.
4. byte_controller.cnt_done remains high once byte_controller.dcnt is 3'b000.

1.When ien rises, irq_flag should be high in the next cycle.
2.irq_flag should remain stable for 5 cycles after ien rises.
3.wb_inta_o should be high only if irq_flag has been true for 3 cycles.
4.wb_inta_o should be deasserted within 1 cycle after irq_flag is deasserted.
5.irq_flag should be high only if ien has been true for 1 cycle.

Figure 4: Prompt and Response Example for description generation in LLM ❷ `Waveform Analyzer`

exercised and also the value propagation among all connected signals; 3) **function**: check if the function defined in the specification is implemented as expected. Fig. 5 demonstrates an example of generating SVAs for a signal.

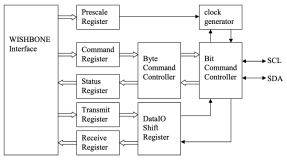
2.5 Evaluation of Generated Assertions

After the SVA generation process, we then evaluate the quality of the generated assertions. While some previous studies such as [8, 38] suggest using specifically-designed checkers for this purpose, such an approach is limited to particular design types like protocols and processors and lacks generalizability to all VLSI designs. Other methods like [26] involve manual inspection by the engineers of FPV results using the generated assertions. While in our approach, we assume there are golden RTL implementations for the designs. Especially, we pick such designs as test cases as they have been thoroughly tested and can be regarded as bug-free golden references.

For evaluation, we also utilize the FPV method. The generated SVAs and the golden RTL designs are fed into a model checker. After performing FPV, we compute the following metrics to evaluate the quality of SVAs: 1) **syntax**: checks if the generated SVAs have syntax errors; 2) **FPV pass/fail**: assuming the RTL designs are bug-free, an SVA that passes the FPV check is considered semantically correct, and conversely, a failure indicates an incorrect SVA. 3) **COI coverage**: Cone of Influence (COI) coverage measures the percentage of design logic that is structurally connected to the properties.

[Prompt for SVA Generator]

- [Upload the architecture diagram] Here is the overall diagram workflow of the design architecture.



- [For each signal and waveform diagram, upload the extracted information and signal mapping] Please generate SVAs for the signal control register (CTR) and waveform descriptions.

[Response from SVA Generator]

Based on the provided specification for the control register (CTR), we can write SVAs to validate its behavior. Let's break down the SVAs according to the types you specified:

```
[width]
assert property (@(posedge wb_clk_i) $bits(ctr) == 8);
[connectivity]
// Check that the I2C core enable bit (Bit 7) properly controls the
EN signal
assert property (@(posedge wb_clk_i) (ctr[7] == 1'b1) |-> (core_en ==
1'b1));
assert property (@(posedge wb_clk_i) (ctr[7] == 1'b0) |-> (core_en ==
1'b0));
// Check that the I2C core interrupt enable bit (Bit 6) properly
controls the IEN signal
assert property (@(posedge wb_clk_i) (ctr[6] == 1'b1) |-> (ien ==
1'b1));
assert property (@(posedge wb_clk_i) (ctr[6] == 1'b0) |-> (ien ==
1'b0));
[function]
// Ensure that changes to the CTR register propagate correctly to EN
and IEN
assert property (@(posedge wb_clk_i) $past(ctr[7]) != ctr[7] |-> ##1
(core_en == ctr[7]));
assert property (@(posedge wb_clk_i) $past(ctr[6]) != ctr[6] |-> ##1
(ien == ctr[6]));
// Assert that the I2C core is disabled on reset
assert property (@(posedge wb_clk_i) (wb_rst_i == 1'b1) |-> (ctr ==
8'h00));
// Assert that the I2C core can be enabled
assert property (@(posedge wb_clk_i) (wb_rst_i == 1'b0 && ctr[7] ==
1'b1) |-> (core_en == 1'b1));
// Assert that interrupts can be enabled
assert property (@(posedge wb_clk_i) (wb_rst_i == 1'b0 && ctr[6] ==
1'b1) |-> (ien == 1'b1));
```

Based on the provided waveform descriptions, we can write SVAs to validate its behavior:

```
[function]
// The assertions generated from the first diagram
assert property (@(posedge wb_clk_i) byte_controller.dcnt ==
3'b000 |-> byte_controller.cnt_done);
assert property (@(posedge wb_clk_i) (byte_controller.dcnt !=
3'b000) |-> (byte_controller.dcnt == $past(byte_controller.dcnt) -
1));
assert property (@(posedge wb_clk_i) (byte_controller.dcnt !=
3'b000) |-> (byte_controller.cnt_done == 1'b0));
assert property (@(posedge wb_clk_i) byte_controller.dcnt ==
3'b000 |-> (byte_controller.cnt_done throughout
(byte_controller.dcnt == 3'b000)));
// The assertions generated from the second diagram
assert property (@(posedge wb_clk_i) ien |-> ##1 irq_flag);
assert property (@(posedge wb_clk_i) ien |-> ##[1:5]
$stable(irq_flag));
assert property (@(posedge wb_clk_i) irq_flag[*3] |-> wb_inta_o);
assert property (@(posedge wb_clk_i) !irq_flag |-> ##1 !wb_inta_o);
```

Figure 5: Prompt and Response Example of LLM ❸ `SVA Generator`

It is a common metric to evaluate the quality and usefulness of the generated properties.

3 CASE STUDY

3.1 Experimental Setup

In our study, the original specification documents are provided in PDF format, including a variety of multi-modal content including text, tables, and figures. The signal definition files and the golden RTL designs are formatted in Verilog. To assess the quality of the generated SVAs, we utilize Cadence JasperGold, one of the leading commercial model checking tools. This evaluation leverages the FPV app in JasperGold to ensure a thorough analysis.

Our experimental setup involves the evaluation of three types of LLMs using our developed generation and evaluation methodology:

(1) GPT-3.5: The freely available commercial version, GPT-3.5 Turbo, supports a context window of up to 16K tokens.
(2) GPT-4o: The state-of-the-art commercial solution, GPT-4o, offers up to 128K token context window and multi-modal capabilities, making it adept at handling the diverse content in the specification documents.
(3) AssertLLM: Customized GPT-4o by incorporating specialized techniques such as RAG and custom instructions, tailoring the models specialized to the SVA generation task.

3.2 Evaluation Metrics

To conduct a thorough evaluation of the generated SVAs, we propose a set of metrics that align with our evaluation methodology. This approach ensures a detailed assessment of the SVAs' quality on both a per-signal and per-design basis.

For each assertion type of an individual signal, our evaluation includes the following metrics: 1) the number of generated SVAs; 2) the number of syntax-correct SVAs; 3) the number of FPV-passed SVAs; 4) COI coverage for all FPV-passed SVAs. We consider an SVA as "passed" if the model checker Jaspergold cannot find any counterexample to it within 5 hours. Furthermore, all SVAs are produced directly from LLMs without any subsequent modifications. Once the evaluation for each signal is complete, we aggregate the statistics of the generated SVAs for each design. We then calculate the proportion of these SVAs that are syntactically correct and can pass the FPV checks. Finally, we calculate the COI coverages for all passed SVAs.

3.3 Assertion Generation Quality

To illustrate the efficacy of AssertLLM, we apply it to an illustrative design case: the "I2C" protocol. The complete specification document for the "I2C" design is structured into six main sections, as discussed in Subsection 2.2. Note that the specification for each signal is unstructured, mainly across the sections like IO ports, registers, and operation. Additionally, we provide the signal definition file containing the IO ports and architectural registers and all the internal wires and registers defined for detailed RTL implementation.

The specification for the "I2C" design uses natural language to define 23 signals, comprising 17 IO ports and 6 architecture-level registers. For the IO ports, we categorize them into 4 types: clock, reset, control signal, and data signal. The architecture-level registers are similarly categorized, based on their functionality, into control and data types. Furthermore, the specification for the "I2C" design utilizes two waveform diagrams to describe the behaviors on 5 different signals. AssertLLM will extract the described behaviors in the waveforms and generate SVAs for these signals.

The evaluation of SVAs generated by our AssertLLM is presented in Table 2. For each signal, we first verify each type of the generated

Table 2: Evaluation of the generated SVAs for design "I2C". AssertLLM generates 65 properties, with 23 for bit-width, 14 for connectivity, and 28 for function. 86% of these generated SVAs are evaluated to be correct both syntactically and functionally.

			AssertLLM				GPT-4o	GPT-3.5
	Signal Type		Assertion Evaluation (#. Generated/#. Syntax Correct/#. FPV Pass)					
			Width	Conectivity	Function	Signal Total	Function	
From natual language	IO (17)	Clock (1)	1/1/1	/		1/1/1	3/1/0	Can not handle the original specification files.
		Reset (2)	2/2/2	/		2/2/2	6/2/0	
		Control (3)	3/3/3	4/4/1	/	7/7/4	9/3/0	
		Data (11)	11/11/11	/		11/11/11	33/11/0	
	Reg (6)	Control (2)	2/2/2	10/10/9	13/13/13	25/25/24	6/2/2	
		Data (4)	4/4/4	/	6/6/4	10/10/8	14/4/4	
From waveform			/		9/9/6	9/9/6	4/4/2	
Design Total			23/23/23★	14/14/10	28/28/23	65/65/56	75/27/8	
			100%/100%†	100%/71%	100%/82%	100%/86%	36%/11%	

★ The results represent the number of generated SVAs, syntax-correct SVAs, and FPV-passed SVAs.
† The percentages indicate the proportion of syntax-correct SVAs and FPV-passed SVAs.

SVAs separately. Then we summarize all the SVAs to provide design-level statistics. All SVAs related to bit-width checking performed correctly. However, a minor portion of connectivity and function SVAs contained errors. For SVAs generated from natural language, the errors are attributed to misinterpretations of the specification or hallucinations of language model. For SVAs generated from waveform diagrams, the failures are mainly due to AssertLLM's inability to infer behaviors not explicitly depicted in the waveform diagrams, resulting in incomplete assertions that do not pass the FPV check. Overall, 86% of the SVAs are both syntactically correct and functionally valid.

In addition to assessing AssertLLM's performance, we also conduct an ablation study to compare the SVA generation capabilities of the original GPT-4o and GPT-3.5 models without the additional techniques, demonstrated in Table 2. When generating SVAs from natural language, the absence of a mechanism for extracting structured signal specifications significantly limits GPT-4o's ability to produce accurate SVAs. Specifically, GPT-4o fails to generate any correct SVAs for I/O ports and only succeeds in creating reset check assertions for registers, resulting in an overall accuracy of just 11%. Furthermore, when generating SVAs from waveform diagrams, the lack of a specialized waveform analysis method leads to fewer assertions and a lower FPV-passing rate. As for GPT-3.5, due to the lack of multi-modal processing capabilities, it cannot generate SVAs directly from the original, multi-modal specification files.

We further examine the COI coverage for various signal-type SVAs, as illustrated in Fig. 6. Our results demonstrate that SVAs generated by AssertLLM achieve high COI coverage, with a total coverage of 93.44% (different types of assertions could cover different parts of the design, so the overall rate is higher than individual ones). Additionally, we observe that SVAs generated for registers exhibit higher COI coverage compared to those for IO signals. This can be attributed to the fact that registers typically connect to more logic elements within the design. However, when using GPT-4o to generate SVAs, the COI coverage only reaches 82.05%, primarily due to its inability to generate a sufficient number of correct SVAs.

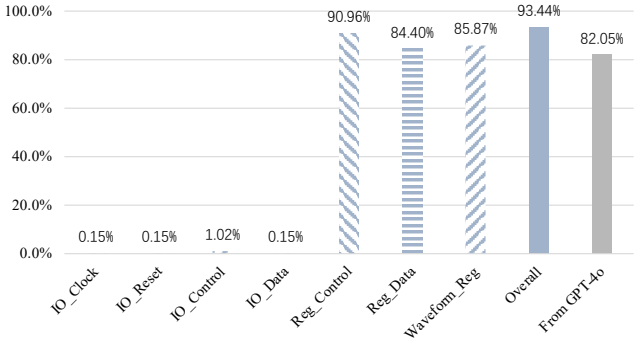


Figure 6: COI coverage of different signal-type SVAs generated by AssertLLM

3.4 Assertion Generation for More Designs

To further analyze the capability of our AssertLLM, we extended its application to generate SVAs for additional designs. In addition to "I2C," we evaluate "ECG" and "Pairing" designs. The "ECG" design calculates the addition of two elements in the elliptic curve group, while the "Pairing" design implements Tate bilinear pairing in the elliptic curve group [9]. Table 3 presents the overall results, demonstrating AssertLLM's ability to generate high-quality SVAs across various designs. All generated SVAs maintain syntactic correctness and a significant proportion of SVAs can pass FPV. Moreover, the FPV-passed SVAs achieve high COI coverage. These indicate good generalizability of our approach. In contrast, GPT-4o struggles to maintain consistent performance across different designs. It fails to generate any FPV-passed SVAs for the "ECG" design and produces only one FPV-passed SVA for the "Pairing" design. Consequently, the COI coverage for GPT-4o's SVAs in these two examples is 0%, as we only calculate COI coverage for FPV-passed assertions.

3.5 Discussion: the Impact of Specification Quality

The generation of high-quality SVAs from natural language specifications relies not only on the capabilities of LLMs but also on

Table 3: Evaluation on more designs.

Design	SVA Evaluation			
	AssertLLM		GPT-4o	
	Total Correctness	Total Coverage	Total Correctness	Total Coverage
I2C	65/65/56 100%/86%	93%	75/27/8 36%/11%	82%
ECG	22/22/20 100%/91%	99%	11/7/0 64%/0%	0%
Pairing	15/15/14 100%/93%	100%	12/8/1 67%/8%	0%
Average	100%/90%	97%	56%/6%	27%

The results share the same format as Table 2.

the intrinsic quality of the specification documents themselves. A specification that provides only the basic information of signals, such as their names and simple descriptions, without delving into detailed functionalities or connectivities, inherently limits the potential for generating meaningful SVAs, regardless of the power of the LLMs employed. Conversely, specifications that offer comprehensive details, including clear definitions of signal functionalities and connectivities, can facilitate the generation of SVAs even with relatively simple LLMs. This observation can be concluded in the Table 3. The "I2C" specification provides detailed information on registers and functionality, enabling the generation of more SVAs. By contrast, both AssertLLM and GPT-4o can only generate a limited number of SVAs for "ECG" and "Pairing" due to their less detailed specifications.

4 CONCLUSION

In this paper, we introduce AssertLLM, an automated framework designed for generating assertions from entire specification documents. AssertLLM breaks down this intricate task into three phases: natural language information extraction, waveform description extraction, and assertion generation, leveraging specialized LLMs for each phase. Experimental results show that AssertLLM generates assertions with an average of 88% passing both syntax-checking and FPV checking. Furthermore, these assertions achieve 97% COI coverage on average, achieving a notable improvement from GPT-4o and GPT-3.5.

ACKNOWLEDGMENTS

This work is supported in part by the National Natural Science Foundation of China under Grant No. 62304194, by Guangzhou Municipal Science and Technology Project under Grant No. 2023A03J0013, and by Hong Kong Research Grants Council (RGC) ECS under Grant No. 26208723.

REFERENCES

[1] Fnu Aditi and Michael S Hsiao. 2022. Hybrid Rule-based and Machine Learning System for Assertion Generation from Natural Language Specifications. In *2022 IEEE 31st Asian Test Symposium (ATS)*. IEEE, 126–131.

[2] Peter Anderson, Xiaodong He, Chris Buehler, Damien Teney, Mark Johnson, Stephen Gould, and Lei Zhang. 2018. Bottom-up and top-down attention for image captioning and visual question answering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 6077–6086.

[3] Jyoti Aneja, Aditya Deshpande, and Alexander G Schwing. 2018. Convolutional image captioning. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 5561–5570.

[4] Kaiyan Chang, Kun Wang, Nan Yang, Ying Wang, Dantong Jin, Wenlong Zhu, Zhirong Chen, Cangyuan Li, Hao Yan, Yunhao Zhou, et al. 2024. Data is all you need: Finetuning LLMs for Chip Design via an Automated design-data augmentation framework. *arXiv preprint arXiv:2403.11202* (2024).

[5] Lei Chen, Yiqi Chen, Zhufei Chu, Wenji Fang, Tsung-Yi Ho, Yu Huang, Sadaf Khan, Min Li, Xingquan Li, Yun Liang, et al. 2024. The dawn of ai-native eda: Promises and challenges of large circuit models. *arXiv preprint arXiv:2403.07257* (2024).

[6] Alessandro Danese, Nicolò Dalla Riva, and Graziano Pravadelli. 2017. A-team: Automatic template-based assertion miner. In *Proceedings of the 54th Annual Design Automation Conference 2017*. 1–6.

[7] Wenji Fang, Guangyu Hu, and Hongce Zhang. 2023. r-map: Relating Implementation and Specification in Hardware Refinement Checking. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2023).

[8] Steven J Frederiksen, John Aromando, and Michael S Hsiao. 2020. Automated Assertion Generation from Natural Language Specifications. In *2020 IEEE International Test Conference (ITC)*. IEEE, 1–5.

[9] Gerhard Frey and Hans-Georg Rück. 1994. A remark concerning m-divisibility and the discrete logarithm in the divisor class group of curves. *Mathematics of computation* 62, 206 (1994), 865–874.

[10] Samuele Germiniani and Graziano Pravadelli. 2022. Harm: a hint-based assertion miner. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 11 (2022), 4277–4288.

[11] Christopher B Harris and Ian G Harris. 2016. Glast: Learning formal grammars to translate natural language specifications into hardware assertions. In *2016 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 966–971.

[12] Rahul Kande, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Shailja Thakur, Ramesh Karri, and Jeyavijayan Rajendran. 2023. LLM-assisted Generation of Hardware Assertions. *arXiv preprint arXiv:2306.14027* (2023).

[13] Rahul Kande, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Shailja Thakur, Ramesh Karri, and Jeyavijayan Rajendran. 2024. (Security) Assertions by Large Language Models. *IEEE Transactions on Information Forensics and Security* (2024).

[14] Oliver Keszocze and Ian G Harris. 2019. Chatbot-based assertion generation from natural language specifications. In *2019 Forum for Specification and Design Languages (FDL)*. IEEE, 1–6.

[15] Rahul Krishnamurthy and Michael S Hsiao. 2019. Controlled natural language framework for generating assertions from hardware specifications. In *2019 IEEE 13th International Conference on Semantic Computing (ICSC)*. IEEE, 367–370.

[16] Rahul Krishnamurthy and Michael S Hsiao. 2019. Ease: Enabling hardware assertion synthesis from english. In *Rules and Reasoning: Third International Joint Conference, RuleML+ RR 2019, Bolzano, Italy, September 16–19, 2019, Proceedings 3*. Springer, 82–96.

[17] Mengming Li, Wenji Fang, Qijun Zhang, and Zhiyao Xie. 2024. SpecLLM: Exploring Generation and Review of VLSI Design Specification with Large Language Model. *arXiv preprint arXiv:2401.13266* (2024).

[18] Mingjie Liu, Minwoo Kang, Ghaith Bany Hamad, Syed Suhaib, and Haoxing Ren. 2024. Domain-Adapted LLMs for VLSI Design and Verification: A Case Study on Formal Verification. In *2024 IEEE 42nd VLSI Test Symposium (VTS)*. IEEE, 1–4.

[19] Mingjie Liu, Nathaniel Pinckney, Brucek Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. *arXiv preprint arXiv:2309.07544* (2023).

[20] Shang Liu, Wenji Fang, Yao Lu, Jing Wang, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2024. RTLCoder: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation Technique. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024).

[21] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2024. Rtllm: An open-source benchmark for design rtl generation with large language model. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 722–727.

[22] Bhabesh Mali, Karthik Maddala, Sweeya Reddy, Vatsal Gupta, Chandan Karfa, and Ramesh Karri. 2024. ChIRAAG: ChatGPT Informed Rapid and Automated Assertion Generation. *arXiv preprint arXiv:2402.00093* (2024).

[23] Ashok B Mehta. 2020. *SystemVerilog Assertions and Functional Coverage*. Springer.

[24] Shunji Mori, Hirobumi Nishida, and Hiromitsu Yamada. 1999. *Optical character recognition*. John Wiley & Sons, Inc.

[25] Marcelo Orenes-Vera, Aninda Manocha, David Wentzlaff, and Margaret Martonosi. 2021. AutoSVA: Democratizing Formal Verification of RTL Module Interactions. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 535–540.

[26] Marcelo Orenes-Vera, Margaret Martonosi, and David Wentzlaff. 2023. Using LLMs to Facilitate Formal Verification of RTL. *arXiv e-prints* (2023), arXiv–2309.

[27] Ganapathy Parthasarathy, Saurav Nanda, Parivesh Choudhary, and Pawan Patil. [n. d.]. SpecToSVA: Circuit Specification Document to SystemVerilog Assertion Translation. In *2021 Second Document Intelligence Workshop at KDD*.

[28] Zehua Pei, Hui-Ling Zhen, Mingxuan Yuan, Yu Huang, and Bei Yu. 2024. Betterv: Controlled verilog generation with discriminative guidance. *arXiv preprint arXiv:2402.03375* (2024).

[29] Vaishnavi Pulavarthi, Deeksha Nandal, Soham Dan, and Debjit Pal. 2024. AssertionBench: A Benchmark to Evaluate Large-Language Models for Assertion Generation. arXiv:2406.18627 [cs.SE] https://arxiv.org/abs/2406.18627

[30] Erik Seligman, Tom Schubert, and MV Achutha Kiran Kumar. 2023. *Formal verification: an essential toolkit for modern VLSI design.* Elsevier.

[31] Liu Shang, Lu Yao, Fang Wenji, Li Mengming, and Xie Zhiyao. 2024. OpenLLM-RTL: Open Dataset and Benchmark for LLM-Aided Design RTL Generation. In *ICCAD*.

[32] Chuyue Sun, Christopher Hahn, and Caroline Trippel. 2023. Towards Improving Verification Productivity with Circuit-Aware Translation of Natural Language to SystemVerilog Assertions. In *First International Workshop on Deep Learning-aided Verification (DAV)*.

[33] Shobha Vasudevan, David Sheridan, Sanjay Patel, David Tcheng, Bill Tuohy, and Daniel Johnson. 2010. Goldmine: Automatic assertion generation using data mining and static analysis. In *2010 Design, Automation & Test in Europe Conference & Exhibition (DATE 2010)*. IEEE, 626–629.

[34] Srikanth Vijayaraghavan and Meyyappan Ramanathan. 2005. *A practical guide for SystemVerilog assertions.* Springer Science & Business Media.

[35] Hasini Witharana, Yangdi Lyu, Subodha Charles, and Prabhat Mishra. 2022. A survey on assertion-based hardware verification. *ACM Computing Surveys (CSUR)* 54, 11s (2022), 1–33.

[36] Haoyuan Wu, Zhuolun He, Xinyun Zhang, Xufeng Yao, Su Zheng, Haisheng Zheng, and Bei Yu. 2024. Chateda: A large language model powered autonomous agent for eda. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024).

[37] Scott Yih. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Conference on Neural Information Processing Systems, Vancouver, Canada*.

[38] Junchen Zhao and Ian G Harris. 2019. Automatic assertion generation from natural language specifications using subtree analysis. In *2019 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 598–601.